Observer

Elisa e Vinicius

1. Introdução ao Padrão: GoF "Observer"

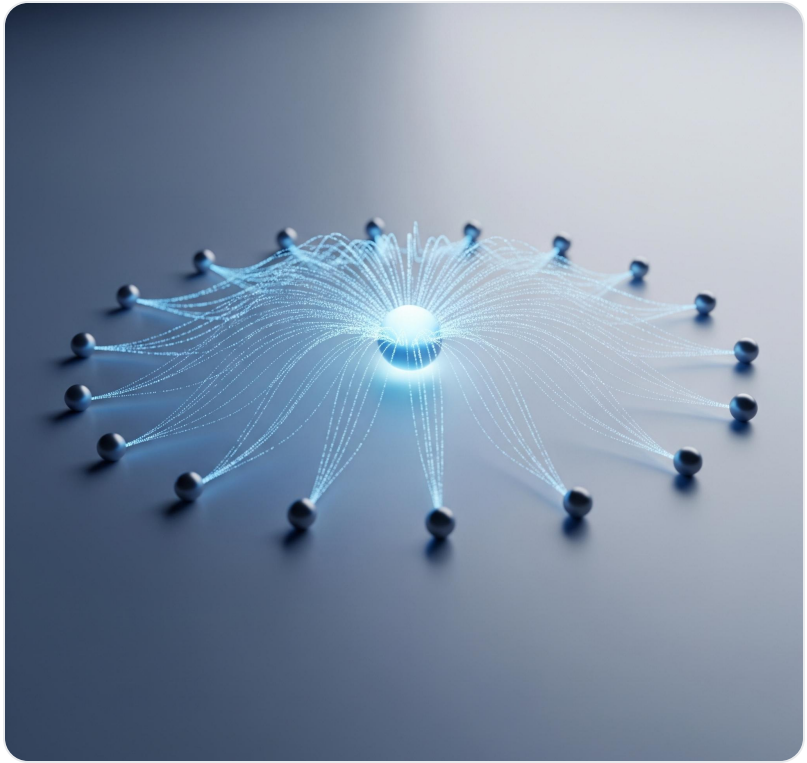
Definição e Propósito

Nome: Observer (Observador)

Intenção: Definir uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes sejam notificados e atualizados automaticamente.

Quando usar: Quando uma abstração tem dois aspectos, um dependente do outro, ou quando uma mudança em um objeto exige mudanças em outros sem saber quantos são.

O padrão Observer, parte do catálogo clássico do **Gang of Four (GoF)**, promove o baixo acoplamento ao permitir que o "Sujeito" gerencie interessados sem conhecer suas implementações concretas.



2. Estrutura e Diagrama: Padrão Observer

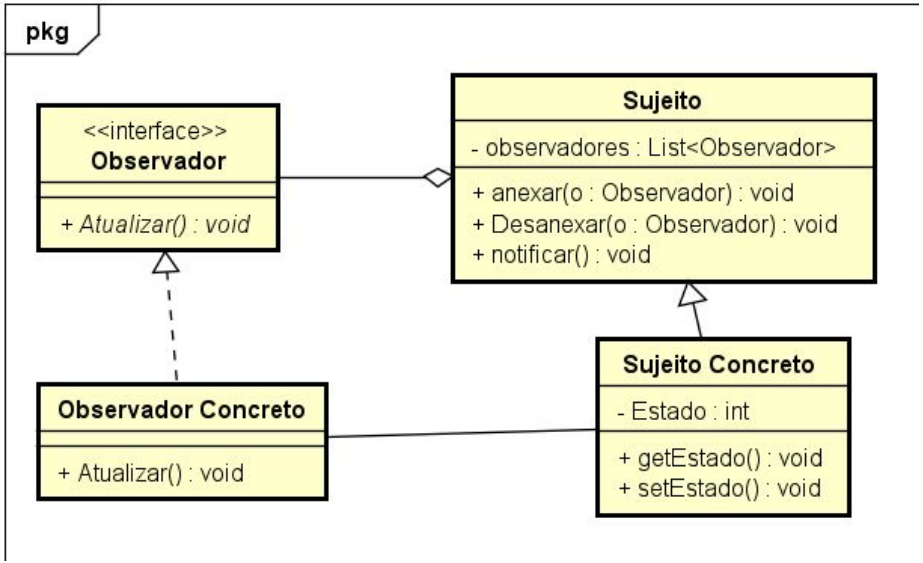
Componentes do Diagrama

Subject (Sujeito): Mantém uma lista de observadores e fornece métodos para anexar, desanexar e notificar.

Observer (Observador): Define uma interface de atualização para objetos que devem ser notificados sobre mudanças no sujeito.

ConcreteSubject: Armazena o estado de interesse para os ConcreteObservers e envia uma notificação quando seu estado muda.

ConcreteObserver: Mantém uma referência para o ConcreteSubject e implementa a interface de atualização para manter seu estado consistente com o do sujeito.



3. Contextualização: O Padrão Observer

3.1 - Problema Real: Notificação de Mudanças

Imagine um sistema de leilão online onde centenas de usuários precisam saber instantaneamente quando um lance é superado.

Sem um padrão, o objeto Leilão teria que manter uma lista rígida e manual de cada usuário, acoplando fortemente o "Sujeito" (o leilão) aos seus "Observadores" (usuários).

3.2 - Por que a Solução Ingênua Falha?

- **Falta de Escalabilidade:** Fazer o Sujeito perguntar a cada objeto se ele quer a atualização gera tráfego de rede e processamento desnecessário.
- **Acoplamento Forte:** O Sujeito precisa conhecer as classes concretas de todos os interessados, dificultando a adição de novos tipos de observadores.
- **Inflexibilidade:** Alterar a lista de interessados exige modificar o código do Sujeito, violando o princípio de aberto/fechado.

4. Implementação Sem o Padrão Observer

4.1. Solução Inicial (Acoplada)

Nesta abordagem, o **Sujeito** possui referências diretas para os observadores, chamando métodos específicos sempre

Java Leilao

```
class class Leilao
  private List<Object>
   A
 1     public  = usuarios {
 2             new UsuarioAndroid('AndroidUser1')')
 3             new UsuarioWeb('WebUser1')')
 8
14         public Leilao()
14
13         public void iniciarLeilao()  {
17                 usuarios(participar((usuartiario.participar));
10                 usuario.participar()
```

4.2 Problemas Identificados

- **Violação do OCP:** Novo observador exige alteração no Sujeito.
- **Dependência Concreta:** O leilão conhece as classes finais.
- **Dificuldade de Testes:** Mocking complexo por falta de interfaces.

5. Implementação com o Padrão Observer

5.1 Refatoração e Estrutura

O objetivo é refatorar a solução anterior para eliminar o acoplamento direto entre o Sujeito e seus Observadores.

Classes Envolvidas:

- **Subject:** Interface que define os métodos de registro e notificação.
- **ConcreteSubject:** O objeto de domínio (ex: Leilão) que notifica mudanças de estado.
- **Observer:** Interface com o método update().

6. Comparação: O Impacto do Padrão Observer

6.1 - Vantagens (O "Depois")

- **Baixo Acoplamento:** O Sujeito não precisa conhecer as classes concretas dos Observadores.
- **Extensibilidade:** Novos observadores podem ser adicionados sem alterar o código do Sujeito (OCP).
- **Comunicação Broadcast:** Notificações automáticas para múltiplos interessados simultaneamente.
- **Reutilização:** Sujeitos e Observadores podem ser reutilizados de forma independente em outros contextos.

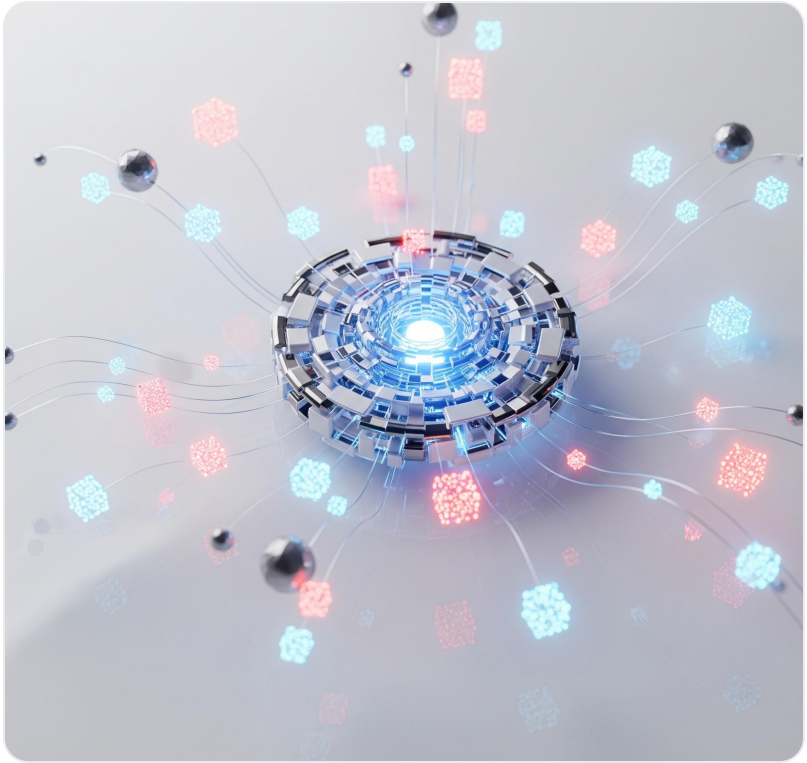
6.2 - Desvantagens & Cuidados

- **Atualizações Inesperadas:** Observadores não têm conhecimento sobre a presença uns dos outros, podendo causar uma cascata de mudanças.
- **Vazamento de Memória:** Se observadores não forem removidos (detach) corretamente, podem impedir a coleta de lixo do Sujeito.
- **Ordem de Notificação:** Não há garantia na ordem em que os observadores serão atualizados.
- **Complexidade Inicial:** Requer a criação de interfaces e infraestrutura de gerenciamento de lista.

7. Conclusão: O Valor do Padrão Observer

Síntese do Aprendizado

O padrão **Observer** consolida-se como uma ferramenta essencial para arquiteturas desacopladas, permitindo que sistemas evoluam de forma sustentável.

- **Flexibilidade:** Adição de novos interessados sem modificar o "Sujeito".
- **Autonomia:** Reduz a dependência entre componentes, facilitando testes e manutenção.
- **Responsabilidade:** Promove o princípio de Responsabilidade Única (SRP) e o Aberto/Fechado (OCP).

Em suma, dominar o Observer transforma o desenvolvedor em um **arquiteto de software** capaz de projetar sistemas escaláveis e confiáveis.